WorkTable

Absolutely not a database.

A user's guide to the `worktable!` macro, its queries, its indexes and its persistence tier. Written against 1.9.0-alpha1.

What this is

Embedded table storage for Rust. You declare a table with a macro and get a typed struct back: a primary key, secondary indexes, and generated queries. Rows live in memory as paged, zero-copy records. Persisting them to local disk or to S3 is opt-in.

If you have used .NET's `DataTable` this will feel familiar. The differences are that the type is generated for you, and that persistence is one feature away.

> **What it is not.** There is no transaction journal and no fsync on every batch. A mutation returning means the change was accepted and queued, not that it is on stable storage. Section 6 says exactly what each boundary guarantees.

Getting started

```
cargo add worktable
```

A table is one macro invocation. The name is the only required key beyond the columns.

```
use worktable::prelude::*;
use worktable::worktable;

worktable! (
    name: Order,
    columns: {
        id: u64 primary_key autoincrement,
        symbol: String,
        quantity: u64,
    },
    indexes: {
        symbol_idx: symbol,
    }
);
```

That generates `OrderWorkTable`, `OrderRow`, `OrderPrimaryKey`, and a `select_by_symbol` method from the index. Nothing is written by hand per table.

```
let table = OrderWorkTable::default();
table
    .insert(OrderRow { id: table.get_next_pk().into(), symbol: "ETH".into(), quantity:
3 })
    .await?;
let found = table.select_by_symbol("ETH".into()).execute()?;
```

Mutations are `async`; reads are not. This declaration and these three calls are compiled and run by `examples/guide_check.rs`, so the guide cannot drift from the API without the build noticing.

Declaring a table

The grammar is positional at the top and block-structured below it. The order is **name, version, persist, partition_by**, and then the blocks `columns`, `indexes`, `queries`, `config`, in any order. Putting `persist` after a block is an error that names the required order rather than failing as an unexpected token.

Columns

Each column is `name: Type` followed by any inline attributes. `primary_key` is required on exactly one column, or on several to form a tuple key. `autoincrement` asks the table to generate the key. `optional` makes the column an `Option`.

Indexes

Each entry is `name: column`, optionally `unique`, optionally `using <backend>`. A non-unique index maps one key to many rows. Every index adds a `select_by_<column>` method.

Queries

Beyond the generated `select`, `insert`, `insert_many`, `upsert`, `update`, `delete` and `select_all`, the `queries` block declares your own update and delete shapes.

> **in_place queries.** An `in_place` query hands you a mutable reference to the archived column bytes and skips index maintenance entirely, so a column covered by any index cannot be mutated that way. The macro refuses it rather than letting an index go stale.

Index backends

An index can name its physical structure with `using`. Four are available and they differ in what they can express, not only in speed.

Backend	When it fits
`worktables_index`	The general one. Takes an ordered key of any type, and the only one that can key an optional or variable-width column.
`indexset`	Vanilla IndexSet, selectable explicitly while keeping the same disk representation.
`arctic`	**The default.** Fixed-width keys only, and the fast one. Packs a row link into a single `u64`.
`congee`	Fixed-width integer keys. Refuses `String` and other variable-width types.

Congee must state `persist: true` or `persist: false` explicitly, because its persistence uses native checkpoint and WAL adapters rather than the shared page format.

Omitting `using` gives `arctic`, with one exception: a composite primary key keeps `worktables_index`, because arctic's key contract cannot represent a tuple.

> **The default cannot key everything.** Arctic takes fixed-width keys only, so an index on an optional or variable-width column must name `worktables_index` explicitly. `by_name: name unique` over a `String optional` is rejected, and the message names the type rather than the omission.

> **Arctic and page size.** Arctic packs a link into 64 bits with 16-bit offset and length fields, so it cannot address a page larger than 65535 bytes. The macro checks this and refuses the combination.

Page size

A page has two sizes and they are not interchangeable. The **stride** is what one page occupies on disk, header included, and every file offset is computed from it. The **inner size** is the stride less the 28-byte header: what a page can actually hold.

Set it in the `config` block:

```
worktable! (
    name: Small,
    columns: { id: u64 primary_key, v: u64 },
    config: { page_size: 4096 }
);
```

> **Persisted tables have a floor, not a fixed size.** `page_size` works for a persisted table, and the only rule is a 512-byte minimum: a page on disk carries a 28-byte header, so anything much smaller is mostly header. An Arctic-backed table is also capped at 65535. In-memory tables have neither limit.
>
> It was refused outright until recently, because the seeks computed offsets from a hardcoded constant while the table threaded the configured one. Every location that decides a page size, and the three silent bugs found while making them agree, are in `docs/page-size.md`.

Columnar fields and indexes

A column marked `columnar` is stored column-wise as well as row-wise, so a scan over that one field reads only that field's bytes instead of walking whole rows.

```
worktable! (
    name: Reading,
    columns: {
        id: u64 primary_key,
        host_id: u64 columnar(chunk_rows(2), compression(none)),
        timestamp: i64 columnar,
        payload: String,
    },
    columnar_indexes: {
        host_time: {
            cluster_by: [host_id, timestamp],
        },
    },
);
```

`columnar` takes optional settings. `chunk_rows(n)` sets how many rows go in a chunk and `compression(name)` selects the codec; `none` is the only one today. A bare `columnar` is not `columnar(...)` with the defaults filled in, and the two are written back differently, so what you wrote is what you get.

`columnar_indexes` names an ordering over columnar fields. `cluster_by` lists the fields, in order, and every one of them must itself be `columnar`. A table with columnar fields and no `columnar_indexes` is fine; the reverse is not.

Two settings live in `config` rather than on a column, because they apply to the table: `columnar_slot_id` picks the width of the slot identifier (`ColumnSlotId8` through `ColumnSlotId64` , default `ColumnSlotId32`) and `columnar_chunk_rows` sets the default chunk size for fields that do not name their own.

> **The primary key is already there.** A primary-key column must not declare `columnar` : it participates in columnar identity implicitly, and declaring it again generates duplicate scan methods. The macro refuses it.

Persistence

Persistence is implemented, not planned. Add `persist: true` and load the table through an engine.

```
let config = DiskConfig::new_with_table_name(dir, OrderWorkTable::name_snake_case(),
OrderWorkTable::version());
let engine = OrderPersistenceEngine::new(config).await?;
let table = OrderWorkTable::load(engine).await?;
```

S3 layers on top of the disk engine rather than replacing it: `S3SyncDiskPersistenceEngine` wraps a `DiskPersistenceEngine` and syncs it. Enable the `s3-support` feature.

The durability contract

This is the part to read before relying on it.

Boundary	What you actually get
A mutation returns	The in-memory change was accepted and its persistence operation was queued.
`wait_for_ops()` returns	The engine completed the queued operations. No fsync, no stable-storage guarantee.
`close()` returns	Intake stopped, the queue drained, the engine task joined. Still no fsync guarantee.
Process crash or `SIGKILL`	Acknowledged rows may be lost and the file may be torn.
Power loss	No atomic-batch or stable-storage guarantee.

Call `close()` during orderly shutdown. `wait_for_ops()` is not a shutdown boundary on its own: it does not stop another task from queueing later work, so it needs application-level writer quiescence to mean anything.

A persistence failure is terminal. An unrecoverable event gap, queue-analysis error, batch-apply error or engine-task failure moves the table into a failed state, and the original error is returned to waiters, to `close()` , and to later mutations.

Loading a torn store

A normal load audits archived rows and both primary and secondary index consistency before exposing the table, and refuses torn state with `PersistenceLoadError` rather than opening plausible-but-invented rows.

`LoadMode::Recovery` exists for offline tools only. It copies individually validated rows through a surviving index into a clean table, which must then pass a normal strict load before anyone reads it. It is not an in-place repair and must never serve live traffic.

Vacuum

Persisted vacuum compacts the in-memory layout and keeps disk indexes consistent with moved rows. It does not truncate `.wt.data`. Watch physical growth with `persisted_data_file_size_bytes().await` and decide when to snapshot and rebuild.

The filesystem

WorkTable reaches the filesystem through one module, `worktable::prelude::fsx`, and names no async runtime. The file type is `std::fs::File` behind `AllowStdIo`, which carries the `futures-io` traits the storage layer asks for while keeping blocking semantics.

That is a deliberate choice and it was measured: `tokio::fs` ran scattered updates at 12,316 rows per second against 74,728 for the same code on `std::fs`, a factor of 6.1, with bulk insert within noise and the in-memory control matching. A scattered update is many small IOs and `tokio::fs` pays a thread-pool round trip for each one.

> **Where to put the work.** Because the calls block, a persistence engine should own a thread rather than share a runtime's worker pool. The calls were never waiting on the disk through a runtime anyway: the persistence path measured 89 voluntary context switches across 25,000 inserts.

Choosing a runtime

A table names the async runtime its generated code awaits on.

```
worktable! (
    name: Orders,
    runtime: nagoya(shared_slot),
    columns: { id: u64 primary_key, total: u64 },
);
```

`nagoya` is the default and `tokio` is the alternative. Omitting `runtime:` and writing `runtime: nagoya(shared_slot)` describe the same table.

The parenthesised name is a **flavor**: a set of scheduler tunings, not a different scheduler. All flavors share one pool implementation, so choosing between them costs no extra code and no rebuild of the engine.

Flavor	What it changes
`shared_slot`	The default. Keeps a self-waking task on its worker, and shares the overflow when more than one piles up behind it.
`locality`	Keeps a self-waking task on its worker and never shares the overflow.
`spread`	Sends every wake through the shared injector instead of keeping it local.
`throughput`	`spread`, taking a larger batch from the injector at a time.
`wide_injector`	`spread`, taking a larger batch still.
`low_latency`	`locality`, looking for work more often before parking.

> **Take the default.** Measured across a read/write mix, YCSB, and a persisted mix, every flavor lands inside the run-to-run noise of every other, on 9 to 16 repetitions per point. The one choice that changes anything is a **negative**: putting a flavor that sends wakes to the injector (`spread`, `throughput`,

> `wide_injector`) on a write-heavy table costs 55% to 57%, because the workload wakes on every await. The default does not do that.
>
> So this is not a knob to tune per table. It is a knob to leave alone unless you have a measurement that says otherwise, and the measurement should report a range rather than a median: a 3-run reading of this reversed twice under 16 runs.

Concurrency

Indexes are lock-free with change-data-capture, and a row-level `LockMap` gives ordered access when you need it. Generated reads always use immutable row-version publication, including in `default-features = false` builds: turning off a Cargo feature must never expose a safe API that races deserialization against page-byte mutation.

Point lookups use a strict backend-specific visibility contract by default. WorkTablesIndex pins the structural mapping until its selected node is locked, so both hits and misses are definitive.

Feature flags worth knowing

Feature	Effect
`std`	On by default. Off, the crate links no `std` and the whole persistence half is gone with it.
`s3-support`	The S3 sync engine, and the HTTP stack under it.
`logical-index-persistence`	Moves unique structural CDC work off the mutation path into the background worker. The page format is unchanged either way.
`wti-predictable-search`	On by default. The branch-based node search, which avoids a measured regression on sequential numeric keys.

The three alternative search policies (`wti-hybrid-search`, `wti-std-search`, `wti-superslice-search`) are compile-time gates. Enable one, and only one, for an unambiguous build. If feature unification turns on several, WorkTablesIndex applies a documented precedence rather than refusing the graph.

Where to look next

Document	Covers
`docs/persistence-durability.md`	The durability contract in full, and the snapshot-restore procedure.
`docs/index-backend-dsl-proposal.md`	The `using` syntax and the capability matrix per backend.
`docs/page-size.md`	Every location across the four crates that decides a page size.
`docs/queries.md`	The generated query surface and the custom query grammar.
`docs/migration.md`	Moving a store between formats.
`docs/known-issues.md`	What is known to be wrong right now.